



SISTEMES OBERTS 2025
PRÀCTICA FRONT-END 2025
SALMA JADIANI MAHMOUH



ÍNDEX

1. INTRODUCCIÓ
2. ESTRUCTURA DE LA PRÀCTICA
3. DECISSIONS DE DISSENY
 - a. MODIFICACIONS EN EL BACK-END
 - b. ESTIL, LOGO
4. JOCS DE PROVES REALITZATS
5. CONCLUSIONS
6. MANUAL D'INSTAL·LACIÓ

1. INTRODUCCIÓ

En aquesta pràctica es desenvolupa una aplicació web que integra completament les funcionalitats de l'API Web RESTfull creada en el primer lliurament. Aquesta aplicació està dissenyada amb l'objectiu de donar una experiència d'usuari rica i interactiva.

Es treballa la distinció entre la part pública i la part privada a través d'un formulari per recollir els credencials de l'usuari. D'entrada, al carregar l'aplicació web es llisten el conjunt d'articles disponibles, tal com la plataforma Medium. Els articles pels quals es requereix identificació per accedir al contingut exclusiu es mostren diferenciats per un candau.

Mitjançant un disseny funcional i en format de graella es mostra la informació rellevant de cada article, amb filtres per tòpics i autors. Clicant a la imatge o al botó "Ver detalls" es fa la redirecció. En cas de que sigui necessari i ja s'hagi iniciat sessió prèviament es fa la redirecció als detalls de l'article; mostrant dades com els tòpics i el text d'aquest. Si es vol accedir a un article privat i encara no s'ha iniciat sessió, el missatge en el botó és "Iniciar sessió per veure detalls". Així doncs, es redirigeix a l'usuari a una pàgina on els seus credencials seran recollits mitjançant un formulari html.

En tot moment l'usuari sabrà si ha iniciat la sessió mitjançant un missatge de benvinguda o l'opció d'iniciar sessió a la part superior de la pantalla. Clicant en el nom en el missatge de benvinguda, l'usuari podrà accedir al seu perfil. En aquesta pràctica també s'han treballat els missatges d'error pels casos on els credencials siguin incorrectes o es tracti de filtrar el llistat d'articles per topics o autors inexistents en la base de dades.

2. ESTRUCTURA DE LA PRÀCTICA

El projecte s'ha implementat seguint una estructura Model Vista Controlador repartit en les carpetes del projecte.

Les crides al **backend** i la gestió de les Responses obtingudes des d'aquestes s'ha fet en Source Packages. Aquí destaquen .service, .model i .controller.

a. .service :

Conté les interfícies i les implementacions que gestionen la **comunicació amb l'API REST**. En aquesta carpeta, es defineixen les operacions que es poden realitzar sobre els articles i els usuaris i la **lògica necessària per gestionar les dades que es reben del servidor**.

La interfície ArticleService inclou *listArticles* i *getPerId*, que permeten obtenir articles filtrats per autor o tema, així com accedir a un article específic mitjançant el seu ID. D'altra banda, la interfície UserService té el mètode *loginCorrecte*, per validar les credencials d'accés i *dadesUsuari* per recuperar la informació associada a l'usuari registrat.

En les implemetacions d'aquestes interfícies s'utilitza WebTarget i els seus mètodes *.path()* per fer les crides HTTP a l'API, gestionant les respostes i les possibles excepcions que puguin sorgir durant el procés.

La separació de responsabilitats i la implementació de les interfícies afavoreix l'escalabilitat de l'aplicació.

b. .model

En aquesta carpeta es desen els objectes utilitzats en l'aplicació. Les entitats llegides en el servei son deserialitzades en aquests objectes per permetre'n la manipulació en aquesta part del frontend. Així doncs hi trobem les mateixes classes que en el backend amb l'addició de AlertMessage per gestionar els missatges d'alerta. La classe ArticleID s'ha implementat per balancejar l'ús del DTO en el backend i tenir l'opció de retornar el text de l'article condicionalment pels casos on es requereixi identificació.

c. .controller

Conté les classes que gestionen les sol·licituds de l'usuari i coordinen la interacció entre les vistes i els serveis. Aquestes classes són fonamentals per MVC, ja que actuen com a **intermediàries entre el model i la vista**, processant les dades i retornant les respostes adequades. Hi trobem doncs: ArticleController, LoginController i UserController, cadascun dels quals s'encarrega d'una funcionalitat específica.

Les anotacions com **@Controller** i **@Path** s'utilitzen per definir les classes com a controladors de recursos i per especificar-ne les rutes d'accés. Per exemple, *ArticleController* gestiona les sol·licituds relacionades amb la visualització d'articles, mentre que *LoginController* s'encarrega del procés d'inici de sessió.

S'utilitza l' **@Inject**, per la injecció automàtica de serveis com *ArticleService* i *UserService*, així com de models per l'*AlertMessage*. Així cada controlador pot accedir als serveis necessaris sense haver de crear instàncies manualment. A més, s'utilitza **@Context** per obtenir l'objecte *HttpServletRequest*, que permet accedir a les dades de la sessió de l'usuari i gestionar les sol·licituds HTTP com afegint-li `.header()` o indicant quin de quin tipus de mètode tracta la sol·licitud.

La crida als `.jsp` de la vista es fa en els returns de les classes controller a partir de les respostes obtingudes i gestionades des del servei. Pels **retorns "alert.jsp"** s'utilitza abans el `models.put()` per guardar els `flashMessage`, i el tipus d'alerta de la classe *AlertMessage* en el model per accedir-hi mitjançant expressió language des del `jsp`. Fent `${alertMessage.text}` i `${alertMessage.type}`.

Pel que fa al **frontEnd**, ens movem a la carpeta `WEB-INF` i `resources`. Aquí destaquen: views i layout i la carpeta `resources` amb css, img.

a. Views

La carpeta `views` conté les pàgines JSP que presenten la interfície d'usuari de l'aplicació, per poder interactuar amb el sistema de manera dinàmica i visual. Cada fitxer JSP mostra informació específica i utilitza les biblioteques JSTL (JavaServer Pages Standard Tag Library) per facilitar la manipulació de dades i la presentació de contingut dinàmic.

- **articles-display.jsp**: Aquesta pàgina és la vista principal que mostra una llista d'articles. Inclou un formulari de filtres que permet als usuaris cercar articles per autor o tema. Cada article es presenta en una targeta que inclou la imatge destacada, el títol, l'autor, un resum, la data de publicació i el nombre de visualitzacions. Pels articles privats es mostra un candau adicional en la tarjeta
En cas que hi hagi errors en la búsqueda, per gestionar el 404 not found s'ha modificat el backend per a que retorni un error 204 No content i s'usa la classe *AlertMessage* per la visualització del missatge corresponent.
- **article-details.jsp**: Aquesta pàgina mostra els detalls d'un article seleccionat. Inclou informació completa incloent el text complet i els tòpics d'aquest. La visualització és comptada i afegida en el recompte de

visualitzacions en un scope d'aplicació. També es proporciona un enllaç per tornar a la llista d'articles, facilitant la navegació de l'usuari.

- **login.jsp:** Aquesta pàgina presenta un formulari d'inici de sessió on els usuaris poden introduir les seves credencials. Inclou un missatge d'alerta per mostrar errors d'autenticació, com ara credencials incorrectes, i un botó per enviar el formulari. Si l'usuari intenta accedir a un article privat sense estar autenticat, se li redirigeix a aquesta pàgina, garantint que la seguretat de l'aplicació es mantingui.
- **perfil-details.jsp:** Aquesta pàgina mostra la informació del perfil de l'usuari, incloent el seu identificatiu i nom. També proporciona un enllaç per tornar a la llista d'articles.

b. Layout

La carpeta layout conté components comuns que es reutilitzen a través de les diferents pàgines de l'aplicació.

- **Alert.jsp:** si hi ha un *flashMessage* disponible, es presentarà en un format d'alerta amb un botó per tancar-lo. Així s'obté una resposta immediata sobre què ha generat la alerta. (Missatge col·locat des del controller amb `models.put()`).
- **Footer.jsp:** dona informació de copyright i dades de contacte del desenvolupador. La seva ubicació fixa a la part inferior de la pàgina garanteix que sigui accessible i no molesti visualment.

c. Css

Conté els fitxers d'estil que defineixen l'aparença visual de l'aplicació. S'utilitza principalment Bootstrap, un framework CSS molt conegut, que ofereix una base sòlida per crear interfícies d'usuari responsives i atractives. A més, s'hi inclouen fitxers CSS personalitzats, com `custom-styles.css`, que permeten ajustar l'estil de l'aplicació d'acord amb les necessitats específiques del projecte. Aquest fitxer facilita l'aplicació d'estils addicionals, com colors, marges, tipografies i altres propietats

d. Img

Conté les imatges utilitzades a l'aplicació, incloent imatges destacades per als articles i les icones.

3. DECISSIONS DE DISSENY

Les decisions de disseny en aquesta pràctica han vingut en gran part condicionades per l'enunciat. A més, s'han realitzat canvis i millores en la part del Back-End per aconseguir les funcionalitats requerides.

L'aplicació s'estructura en un **Model Vista Controlador**, permetent tenir així tenir la lògica de negoci separada de la presentació. Les crides es fan al BackEnd des de les classes del servei fent les crides a les `.path()` definides.

a. MODIFICACIONS EN EL BACK-END

- Addició de la relació entre les entitats **Usuari i Credencials @OneToOne** mapejada amb l'id dels credencials (FK) a Usuari
- En la classe Usuari s'han implementat els mètodes `serializable hashCode`, equals i `toString`, permetent **simplificar les consultes SQL** en `ArticleFacadeREST()`.
- S'ha modificat la manera de **verificació de les credencials**. Deixant la classe de filtre funcional, enlloc de fer la comprovació mirant el nom del mètode s'ha optat per una opció més escalable. Es passa a fer la comprovació pels mètodes que la necessitin (`get article id`) mitjançant el mètode **`ValidarHeader()`**.

```
private boolean validarHeader(HttpHeaders headers) {  
    List<String> headersAuth = headers.getRequestHeader(HttpHeaders.AUTHORIZATION);
```

Aquí s'extreuen les capçaleres `Authorization` i es decodifiquen obtenint així l'usuari i la contrassenya amb que es fa la sol·licitud. Mitjançant una consulta es busca en la base de dades amb la `NamedQuery` proporcionada a l'esquelet de la pràctica i es verifica que la contrassenya sigui correcta per l'usuari. La implementació d'aquest mètode requeriria la relació `OneToOne` mencionada en el primer apartat.

- S'ha afegit un **endpoint adicional /validate** a la classe `credencials` per a retornar un codi 200 OK en cas que les credencials estiguin en la taula `credencials`. Aquest mètode aprofita el filtre utilitzant el `@Secured`.

```
@POST  
@Path("/validate")  
@Secured  
public Response validate(){ //si s'hi pot accedir es que la capçalera és vàlida.  
    return Response.ok().build();  
}
```

D'aquesta manera, generar la sessió en el front-end es basarà en una crida des del servei a `/validate` amb les credencials codificades i posades a la

capçalera amb `.header()`. En cas que es pugui fer el post (OK 200) l'usuari està validat i es pot generar des del controller la sessió amb `.setAttribute()`

- Modificacions **install.jsp**. Els canvis esmentats anteriorment relacionats amb el mapeig i les taules s'han vist reflectits en els inserts a la base de dades.

b. Recursos externs

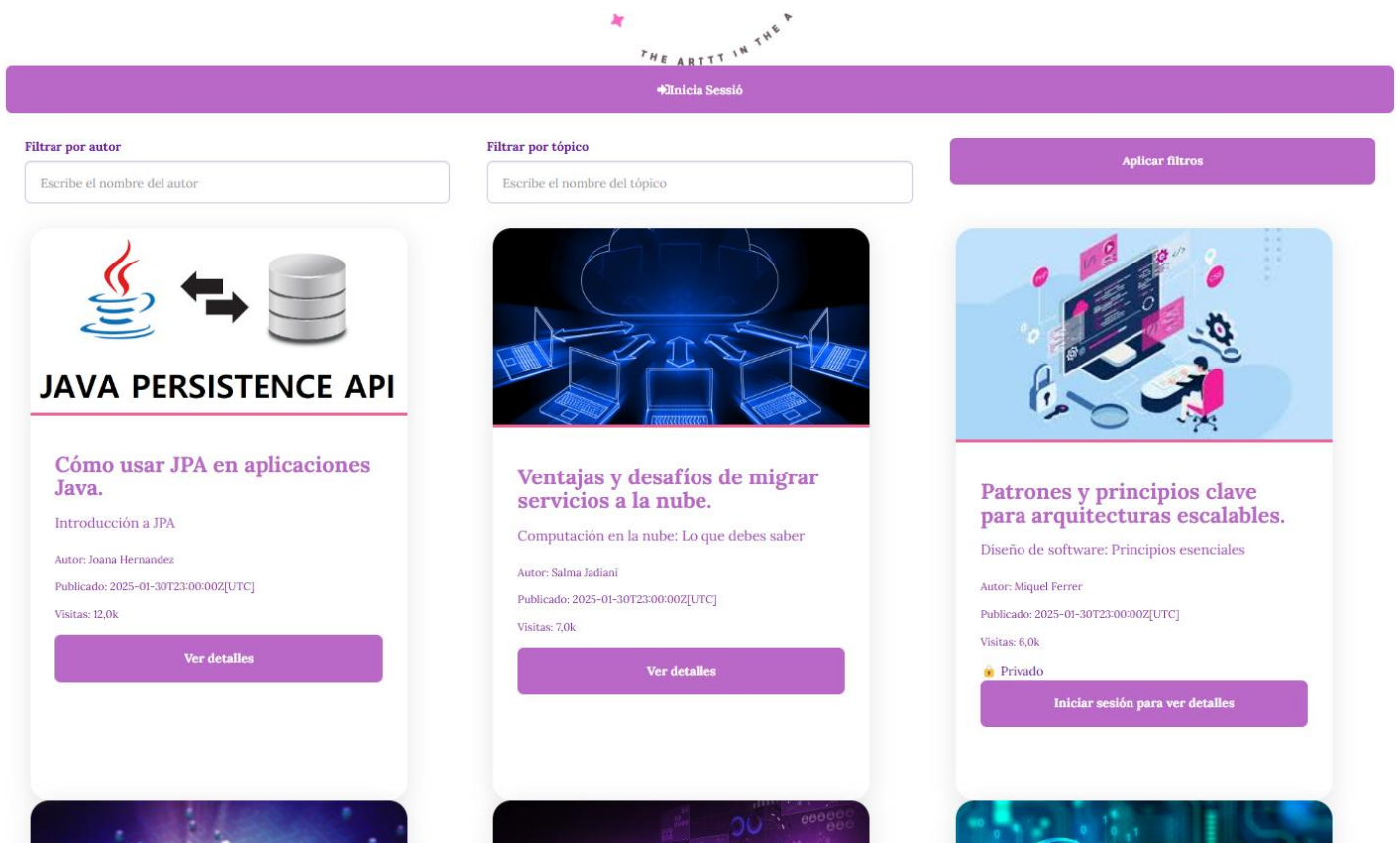
S'ha fet un import de la font de google Lora per als textos que apareixen en la web. A part, les imatges es troben desades en la carpeta img. El disseny del loco s'ha fet a canva.

4. JOCS DE PROVES REALITZATS

-articles-display.

Aquí s'ha provat la crida a autors, topics existents i inexistents. Esperant llistar-los –grid ordenats per views en el primer cas i que aparegui el missatge d'error en el segon.

1. Sense especificar:



The screenshot displays a web application interface with a purple header bar containing a login button labeled "Inicia Sessió". Below the header, there are two filter input fields: "Filtrar por autor" and "Filtrar por tópic", each with a placeholder "Escribe el nombre del autor" and "Escribe el nombre del tópic" respectively. To the right of these filters is a purple button labeled "Aplicar filtros". The main content area shows a grid of three article cards. Each card has a header image, a title, a subtitle, author information, publication date, and a "Ver detalles" button. The first card is titled "JAVA PERSISTENCE API" by Joana Hernandez. The second card is titled "Ventajas y desafíos de migrar servicios a la nube." by Salma Jadiani. The third card is titled "Patrones y principios clave para arquitecturas escalables." by Miquel Ferrer. The interface also includes a "Privado" status indicator and a button to "Iniciar sesión para ver detalles" on the third card.

2. Per autor

•Inicia Sessió

Filtrar por autor

Mery Gomez

Filtrar por tópic

Escribe el nombre del tópic

Aplicar filtros



Principales amenazas y cómo protegerte en línea.
Ciberseguridad en la era digital

Autor: Mery Gomez
Publicado: 2025-01-30T23:00:00Z[UTC]
Visitas: 3.0k

Privado

Iniciar sesión para ver detalles



En este artículo se habla sobre WebAssembly y cómo esta tecnología está revolucionando el desarrollo web.
WebAssembly y su evolución en el desarrollo web.

Autor: Mery Gomez
Publicado: 2025-01-30T23:00:00Z[UTC]
Visitas: 10k

Privado

Iniciar sesión para ver detalles



En este artículo se habla sobre las carteras digitales y la historia del Bitcoin.
El pasado de la moneda del futuro.

Autor: Mery Gomez
Publicado: 2025-01-30T23:00:00Z[UTC]
Visitas: 80

Privado

Iniciar sesión para ver detalles

3. Per topic

Filtrar por autor

Escribe el nombre del autor

Filtrar por tópic

Cloud Computing

Aplicar filtros



Ventajas y desafíos de migrar servicios a la nube.
Computación en la nube: Lo que debes saber

Autor: Salma Jadiani
Publicado: 2025-01-30T23:00:00Z[UTC]
Visitas: 7.0k

4. Error

Filtrar por autor

Escribe el nombre del autor

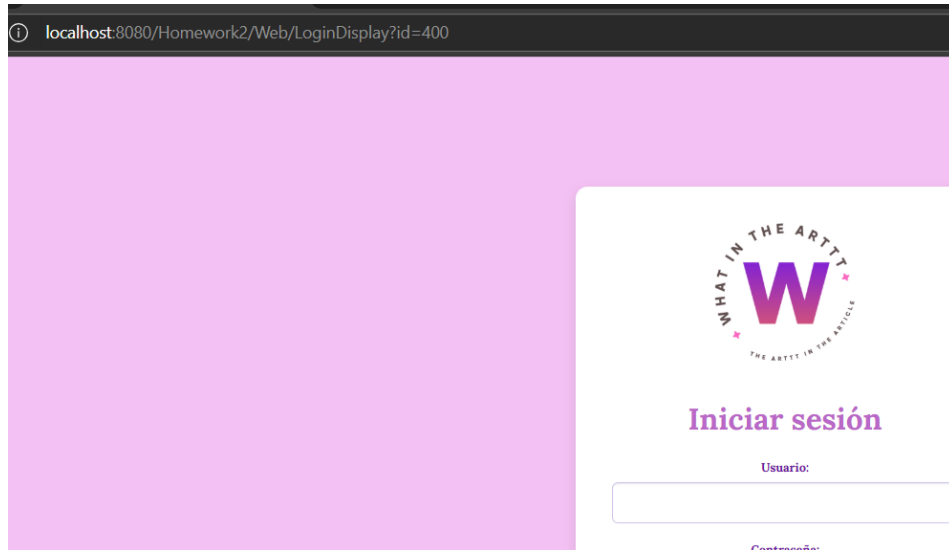
Filtrar por tópic

Error

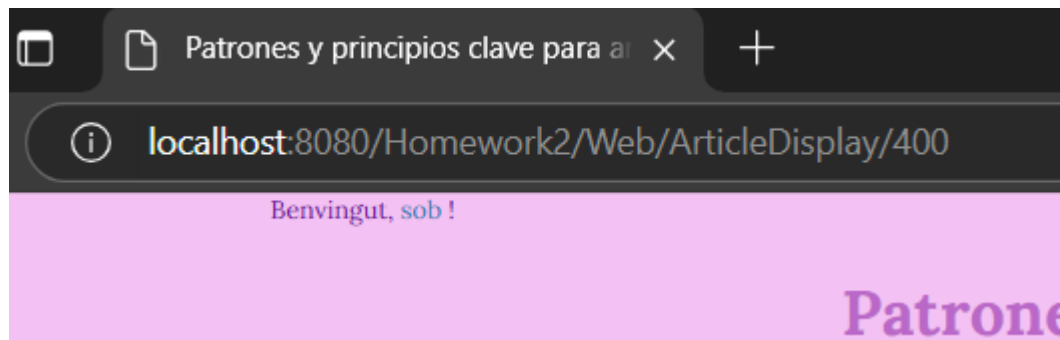
No hi ha articles per l'autor o tòpic.

-articles details. S'espera que es faci una redirecció al login pels articles privats si l'usuari no està autenticat (passant l'id al URL) Si ho està, el botó porta al detall.

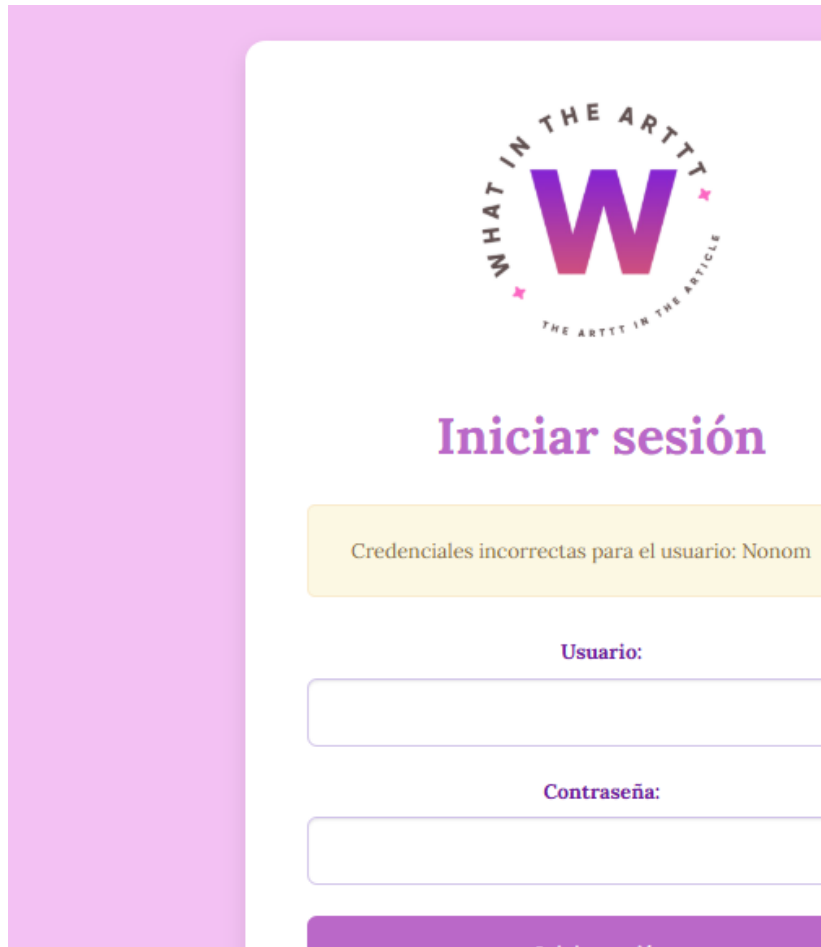
1. id al url i passem al login



2. detalls de l'article privat quan s'ha logejat



-login : si les credencials son incorrectes, s'espera un missatge d'error.



The login form features a pink background on the left and a white area on the right. At the top right is a circular logo with a purple 'W' and the text 'WHAT IN THE ARTTT' and 'THE ARTTT IN THE ARTICLE'. Below the logo is the heading 'Iniciar sesión'. A yellow message box states 'Credenciales incorrectas para el usuario: Nonom'. There are two input fields labeled 'Usuario:' and 'Contraseña:'. A purple button is at the bottom right.

-perfil:



The profile page has a light gray background. It features the same circular logo at the top. Below it is the heading 'Mi perfil'. The user's details are listed: 'Identificatiu: 3' and 'Nom: Miquel Ferrer'. A blue link 'Volver a los artículos' is present. At the bottom, the copyright information reads 'Copyright@ Salma Jadiani salmajadiani@estudiants.urv.cat'.

5. CONCLUSIONS

Sento que en aquesta pràctica hem après a treballar amb les eines bàsiques i fonamentals del que es la part del Front End de la carrera, Buscant informació a internet, he contrastat i vist que tot i que avui en dia existeixen molts frameworks que faciliten la part de disseny de les pàgines és essencial dominar el trio de HTML, CSS i JavaScript. En aquesta pràctica també ens hem familiaritzat amb postman i conceptes d'autentització i com gestionar els missatges d'alerta. Sento que aquests recursos són una pinzellada que hem donat però aquesta segona part ha deixat ficar una part de la nostra creativitat.

Els elements que més m'han agradat:

1. Mapeig de JPA: El mapeig de JPA ha estat realment interessant perquè ens ha permès llegir d'una base de dades real. El fet de poder veure les taules quan s'omple i donar-li al refresh i veure canvis ha sigut com treballar des dels nivells més baixos l'app. Automatitzar la persistència de dades de manera amb codi SQL i treballar des de la façade del servei ens ha deixat veure aquest paper intermediari de les APIs tant essencial.
2. CSS i el disseny visual: M'agrada molt com CSS pot transformar la presentació d'una pàgina basant-nos en la classe de l'objecte o si fa falta en l'identificador mateix. He jugat amb les maneres de donar estil ja que a part de tenir el meu custom-style.css inclòs en tota la web ens alguns casos he usat l'etiqueta style. El fet d'haver de canviar la lògica dona lloc a joc i proves.
3. Ús de JSP per a la presentació dinàmica: La creació de pàgines JSP amb dades dinàmiques ha estat una de les parts més útils, fent les crides al controlador i mitjançant el retorn d'aquests s'ha treballat el redireccionament.

6. MANUAL D'INSTAL·LACIÓ

- a. Build, clean run de Homework1
- b. Insert del client a les bases de dades
- c. Build, clean run de Homework 2.